Graduation Project I

Evaluation-Driven Adaptive Agent Systems

Literature Review and Proposed Methodology

Mohamed Yasser
Belal Sherif

October 2026

This document presents a critical literature review and an experimental proposal for Graduation Project II. Numerical results quoted in it are published findings of prior studies. The proposed system has not yet been implemented or evaluated, and no result of the proposed experiments is reported or assumed.

Abstract

Tool-using language-model (LM) agents depend both on the *harness* that surrounds the model (instructions, demonstrations, tool descriptions) and on the *model* chosen to execute each task. Prior work improves these two decisions largely in isolation: prompt and workflow optimizers adapt the harness for a fixed model, while routers select models for a fixed system. This project asks whether a single evaluation layer can supply the evidence for both adaptations, and whether the two interact.

A focused critical review of 25 primary studies traces a chain from environment-grounded evaluation and failure attribution, through textual and workflow-level optimization, to query-level and agentic model routing. The review finds strong support for each component but limited controlled evidence on how offline harness adaptation and task-level routing behave when trained from the same evaluator and assessed on held-out, stateful tool tasks.

We propose a sandboxed order-support benchmark with deterministic success and policy checks, an audited trace-diagnosis layer, a bounded reflective harness optimizer, and an offline two-model task router. Four configurations are compared: (A) fixed harness and fixed model, (B) optimized harness, (C) dynamic routing, and (D) both. Inexpensive derived controls (a random proportion-matched router, an oracle router, and a frozen pre-adaptation router) separate genuine task-conditioned routing from simple changes in strong-model usage. Development, validation, and test tasks are separated; each test task is repeated five times; and success, policy compliance, cost, latency, reliability, and offline adaptation overhead are reported with paired cluster-bootstrap intervals. No improvement over the baseline is assumed.

Keywords: LLM agents; agent evaluation; failure attribution; prompt optimization; model routing; constrained multi-objective evaluation.

Contents

Abstract i

1 Introduction 1

2 Problem Definition and Objectives 1
2.1 Problem Statement and Research Questions . . . 1
2.2 Measurable Objectives . . . 1
2.3 Scope . . . 1

3 Technical Background and Key Concepts 2
3.1 Agent, Harness, and Execution Trace . . . 2
3.2 Two Distinct Adaptation Decisions . . . 2

4 Literature Review 2
4.1 Evaluation of LLM Agents . . . 3
4.1.1 Environment-Grounded Outcomes and Repeated Reliability . . . 3
4.1.2 Trajectory Evaluation and Judge Reliability . . . 3
4.1.3 Implications for This Project . . . 3
4.2 Failure Attribution and Learning Signals . . . 4
4.2.1 From Observed Error to Localized Diagnosis . . . 4
4.2.2 A Trace-to-Adaptation Interface . . . 4
4.3 Automatic Harness and Agent-System Optimization . . . 4
4.3.1 Foundations: Learning from Textual Feedback . . . 4
4.3.2 Metric-Guided Program Optimization: DSPy and MIPRO . . . 5
4.3.3 Reflective Optimization: GEPA and SIMBA . . . 5
4.3.4 Beyond Prompts: Automated Agent and Workflow Design . . . 5
4.3.5 Critical Comparison and Design Choice . . . 6
4.4 Dynamic Model Routing . . . 6
4.4.1 Cascades and Escalation . . . 6
4.4.2 Learned Predictive Routing . . . 6
4.4.3 Routing Inside Agents . . . 7
4.5 Multi-Objective Evaluation . . . 7

5 Synthesis of Prior Work and Research Gaps 7
5.1 Cross-Study Comparison . . . 7
5.2 Research Synthesis . . . 7
5.3 Bounded Research Gaps . . . 9

6 Proposed Solution and Methodology 10
6.1 Architecture and Adaptation Boundary . . . 10
6.2 Benchmark and Baseline Agent . . . 10
6.3 Evaluation Engine and Diagnostic Audit . . . 11
6.4 Harness Adaptation . . . 11
6.5 Offline Routing Policy . . . 11
6.6 Four-Configuration Experiment . . . 12

7 Technical Requirements and Reproducibility 13

8 Experimental Validity and Analysis Plan **14**
8.1 Data Separation and Fair Selection 14
8.2 Metric Definitions and Resource Accounting 14
8.3 Repeated Runs, Uncertainty, and Failure Reporting 15

9 Expected Contribution and Positioning **15**

10 Limitations and Risks **15**

11 Conclusion **16**

1 Introduction

LM agents complete tasks through sequences of model decisions and tool interactions. Their behavior depends on instructions, demonstrations, tool documentation, state management, and execution limits, so replacing the underlying model is only one way to improve an agent. DSPy showed that the textual parameters of an LM program can be optimized against a downstream metric, and reflective methods such as GEPA use execution evidence to propose prompt changes [11, 13]. Broader work automatically searches whole agent designs and workflows [15, 17].

Reliable execution nevertheless remains difficult. On τ-bench, strong function-calling agents succeed on fewer than half of policy-constrained tasks and are markedly less consistent across repeated attempts [1]. A second difficulty is resource allocation: a strong model may be unnecessary for some requests, while a cheaper model fails on others. FrugalGPT, RouteLLM, and MixLLM address this through cascades and learned routing [20–22], and recent benchmarks show that routing inside multi-step agents differs from routing single queries [24, 25].

These directions raise a common question: *can the evidence collected to improve an agent's behavior also determine which model should perform a task, and do the two adaptations help or interfere with each other?* This thesis studies that question through controlled offline adaptation rather than unrestricted self-modification. Section 2 defines the problem; Sections 3–5 review and synthesize prior work; Sections 6–8 specify how the system will be built, run, and compared in Graduation Project II.

2 Problem Definition and Objectives

2.1 Problem Statement and Research Questions

A fixed harness and a fixed model allocation can be inefficient across tasks with different requirements. Manual prompt revision offers little experimental control, and routers trained on generic response preferences may not predict successful, policy-compliant tool execution. The problem is to determine whether shared outcome and trajectory feedback can improve both decisions without hiding quality loss behind lower cost.

Main research question. How effectively can offline evaluation feedback improve the success–cost–latency trade-off of a tool-using agent through harness adaptation, model routing, and their combination?

RQ1 Which outcome checks and trajectory diagnoses provide reliable and *actionable* feedback for harness adaptation?

RQ2 Can a task-level router reduce inference cost while satisfying a predeclared success requirement, and does it outperform routing that merely matches its strong-model usage rate?

RQ3 Does combining harness adaptation and routing improve the attainable trade-off beyond either mechanism alone, after accounting for adaptation cost, and must the router be re-fitted after the harness changes?

2.2 Measurable Objectives

2.3 Scope

The project studies one controlled, stateful tool domain, a two-model pool, and batch adaptation between evaluation rounds. The router chooses one model before each task and keeps it for the whole run; the routing *policy* changes offline while model selection remains dynamic across tasks. Turn-level switching, production deployment, model-weight training, and changes to

Table 1: Project objectives and the evidence that will verify them. All thresholds are fixed before final testing.

ID	Objective	Evidence required
O1	Build a repeatable stateful tool-agent benchmark.	Resettable state, executable goal predicates, versioned task splits, complete run records.
O2	Validate outcome and diagnostic evaluation.	Checker fixtures; diagnostic precision, recall, and inter-rater agreement on human-labelled traces.
O3	Assess automatic harness adaptation.	Held-out success and violation differences (B vs. A); scalar vs. structured-feedback ablation.
O4	Assess cost-aware routing.	Success and cost differences (C vs. A); comparison with random proportion-matched and oracle routing.
O5	Assess combined adaptation.	Four-configuration comparison, interaction estimate, re-fitted vs. frozen router, adaptation overhead and break-even.

executable tools are outside the primary experiment. Automatic redesign of the agent's workflow structure [15, 17] is related work that is deliberately not attempted, because a narrow textual search space makes behavioral changes attributable.

3 Technical Background and Key Concepts

3.1 Agent, Harness, and Execution Trace

An *agent* maps a task and its observations to a sequence of actions; an action either requests a tool operation or terminates with a response. The environment returns tool results and updates its state. A run therefore has both an *outcome* (the final state and response) and an observable *trajectory*, following the evaluation settings of τ-bench and AgentRewardBench [1, 3].

The *harness* is the configuration and control logic surrounding the model. The *optimizable* harness in this project is narrower: system instructions, demonstration selection, and tool-description wording. Tool implementations, permissions, policy constraints, and evaluator code are fixed, so an optimizer cannot raise its score by weakening the task or its checks.

A *trace* records the task identifier, configuration version, selected model, observable messages, tool calls with arguments and results, timestamps, token usage, and terminal status. Private model reasoning is not required; diagnoses must cite observable evidence.

3.2 Two Distinct Adaptation Decisions

Let h denote the harness and $r(x) \in \mathcal{M}$ the model chosen for task x. Harness adaptation changes *how* the agent is instructed; routing changes *which* model executes it. The two effects need not be additive: a revised prompt can change the relative suitability of models, so a router fitted for one harness may be miscalibrated for another. The combined system must therefore be evaluated directly. A shared evaluator supplies both processes: diagnoses support harness edits, while per-model outcomes and costs support router training. Sharing evidence does not mean sharing features, and no information unavailable at deployment time may reach the router.

4 Literature Review

Review scope and method. This is a focused critical review, not a systematic review. Ten core studies were selected because they map directly onto the proposed chain *evaluate* $\rightarrow$ *diagnose*

→ *adapt harness* → *route model*. Forward and backward citation searches from these papers, plus targeted searches for 2025–2026 work on agent-system optimization, failure attribution, and agentic routing, added fifteen complementary studies. Inclusion required (i) a method or benchmark directly usable by, or directly competing with, a component of the proposed system and (ii) an archival venue or, for 2026 work, a public preprint, which is labelled as such. Bibliographic records were checked on 16 October 2026; software documentation is used only for implementation details [26]. Results from different tasks, metrics, models, and budgets are treated as separate evidence, never as a common leaderboard.

4.1 Evaluation of LLM Agents

4.1.1 Environment-Grounded Outcomes and Repeated Reliability

τ-bench evaluates agents that converse with an LM-simulated user and call database-backed tools under written domain policies. Success is determined by comparing the final database state with the annotated goal state. In the original study, strong function-calling agents succeed on fewer than 50% of tasks, and retail pass^8 falls below 25% [1]. The pass^k metric asks whether *all* k independent attempts succeed, separating occasional from consistent success; a system with acceptable mean success may still fail repeatedly on a subset of tasks. These figures describe the paper's models and benchmark version, not current models.

ToolSandbox complements this design with stateful tool execution, implicit state dependencies between tools (for example, a tool that only works after a setting has been enabled), an LM-simulated user for on-policy conversation, and dynamic evaluation of intermediate and final *milestones* [2]. Its tasks show that state dependencies, canonicalization of arguments, and situations with insufficient information remain difficult even for strong models.

The two benchmarks differ in what they reward. Final-state comparison is unambiguous when one correct end state exists but may reject valid alternatives or miss side effects outside the compared fields; milestone matching credits intermediate progress but requires more annotation. The proposed evaluator borrows from both: goal predicates over the final state, independent invariants that detect unauthorized side effects, and repeated runs that measure consistency.

4.1.2 Trajectory Evaluation and Judge Reliability

AgentRewardBench shifts attention from agent capability to evaluator accuracy. It contains 1,302 expert-reviewed web trajectories from four agents across five benchmarks, labelled for success, side effects, and repetition, and compares twelve LLM-judge configurations; no judge performs best on all benchmarks [3]. In aggregate, the official rule-based evaluators reach 83.8% precision and 55.9% recall (F1 67.1%), whereas a simplified GPT-4o judge reaches 69.8% precision and 83.1% recall (F1 75.9%). The higher F1 comes with lower precision: replacing rules with an LLM changes which errors are made rather than uniformly improving evaluation.

For an adaptive system this matters asymmetrically. A false positive rewards behavior the optimizer will then reinforce, whereas a false negative merely withholds credit. In a structured API environment, state changes and schema validity can be checked directly, so LLM-based judgment is reserved for *explaining* failures, never for deciding success.

4.1.3 Implications for This Project

The reviewed evaluation work supports a layered contract: *task success* is a goal predicate; *policy compliance* is a separate requirement; *trajectory defects* explain unsuccessful or inefficient behavior; and *reliability* is measured across repeats. A successful run with a policy breach remains unacceptable, and a successful run with a redundant loop remains an efficiency defect. Keeping these dimensions separate preserves useful feedback without collapsing them into one opaque score.

4.2 Failure Attribution and Learning Signals

4.2.1 From Observed Error to Localized Diagnosis

Who&When formalized automated failure attribution: given a failed multi-agent log, identify the responsible agent and the decisive step [4]. Across logs from 127 multi-agent systems, the best evaluated method identified the responsible agent in 53.5% of cases but the exact failure step in only 14.2%. The result establishes that step-level localization is substantially harder than coarse attribution.

AgentRx addresses this by synthesizing constraints from the execution context, checking them along the trajectory, and passing an auditable violation log to an LLM judge that predicts the critical step and failure category [5]. The reviewed version (arXiv v2) contains 170 failed trajectories across eleven settings; on its τ-bench subset, AgentRx reaches $42.7 \pm 1.5\%$ exact-step accuracy against $18.8 \pm 5.3\%$ for an adapted Who&When baseline (mean $\pm$ SD over three runs). The EMNLP 2026 publication record describes a 115-trajectory benchmark; numbers in this thesis are taken only from v2, and the camera-ready version will be checked before final submission.

Two 2026 preprints begin to connect diagnosis with improvement. DeFA builds an event-dependency graph to trace how suspect events propagate into failure, reports the best agent- and step-level attribution on Who&When across tested backbones, and reports that feeding its diagnoses into a skill-evolution pipeline raises downstream accuracy by 6–15 percentage points [6]. AgentTether uses graph-guided diagnosis either offline or as a runtime repair layer on τ-bench-style tasks [7]. Both are recent and not yet peer reviewed; they motivate, but do not settle, the question studied here.

Critical comparison. Who&When and AgentRx measure whether a diagnosis matches human labels; DeFA and AgentTether additionally measure downstream effects, but under online intervention or a different adaptation pipeline. None tests whether structured diagnoses improve *offline harness optimization* relative to scalar outcome feedback under a matched budget. Diagnostic accuracy and diagnostic usefulness are distinct properties.

4.2.2 A Trace-to-Adaptation Interface

The project derives a compact feedback record rather than copying a cross-domain taxonomy. Each record contains a task and step reference, the violated requirement, an evidence excerpt, a proposed category, and a status (*confirmed* or *inconclusive*). Operational categories are: wrong tool or arguments, missing prerequisite, repeated action, premature termination, policy breach, and failed recovery. The first visible error, the first unrecoverable error, and the terminal failure can be different events, so a corrected invalid call is not automatically labelled the cause.

Each category maps to a permitted *kind* of edit: a missing prerequisite may motivate an instruction to inspect state before mutation; repeated calls may motivate a stopping rule; ambiguous tool choice may motivate clearer tool descriptions. The mapping only proposes candidates: no diagnosis can change tool semantics or relax a policy, and every candidate is validated by re-running tasks.

4.3 Automatic Harness and Agent-System Optimization

4.3.1 Foundations: Learning from Textual Feedback

Reflexion showed that agents can improve across attempts without weight updates by storing verbal self-reflections on task feedback and conditioning later attempts on them [8]. This is *episodic* adaptation: reflections help retries of the same task. Prompt optimizers instead seek a single prompt that transfers to new inputs. ProTeGi derives natural-language "gradients" from

minibatch errors, edits the prompt accordingly, and searches candidates with beam search and bandit selection, improving initial prompts by up to 31% on its tasks [9]. OPRO uses an LLM as a black-box optimizer that proposes new instructions conditioned on previous instructions and their scores, reporting gains of up to 8% on GSM8K and up to 50% on Big-Bench Hard over human-written prompts [10]. These methods establish that failure-driven textual optimization predates the agent setting; they were evaluated mainly on single-call tasks.

4.3.2 Metric-Guided Program Optimization: DSPy and MIPRO

DSPy represents LM pipelines as declarative modules whose prompt parameters are optimized against an end-to-end metric; its original compiler bootstraps and selects demonstrations, with case studies including agent loops [11]. Its key contribution here is the separation of program structure, metric, and optimizable text. MIPRO extends this to multi-stage programs with only downstream supervision: it proposes instructions grounded in program and data context and searches instruction–demonstration combinations with minibatch evaluation and a surrogate model, outperforming comparison optimizers in five of seven settings (by up to 13 percentage points) [12]. MIPROv2 is the later DSPy implementation of this lineage [26]; implementation behavior is not attributed to the original paper.

4.3.3 Reflective Optimization: GEPA and SIMBA

GEPA samples trajectories, reflects on them in natural language to diagnose failures, proposes prompt updates, and keeps complementary candidates on a Pareto front of per-instance scores. Across six tasks it reports a mean advantage of six percentage points over GRPO (maximum nineteen), up to 35× fewer rollouts, and an advantage of more than ten points over MIPROv2 in aggregate [13]. Its feedback path matches the proposed trace interface closely. However, rollout count is not monetary cost: reflection calls, candidate evaluation, and longer prompts add tokens. GEPA's Pareto front is also over *instances*, not over success, cost, and latency.

Du et al. compare COPRO, MIPROv2, and SIMBA for tabular fact verification under direct, chain-of-thought, ReAct, and CodeAct prompting [14]. Effects depend on prompting style and model scale: on SciTab ReAct, SIMBA raises Qwen3-32B accuracy from 61.5% to 66.2% but lowers Qwen3-8B from 61.3% to 58.3%. The ReAct setting uses a single SQL tool, so evidence about choosing among competing tools remains limited.

4.3.4 Beyond Prompts: Automated Agent and Workflow Design

A separate line of work searches over entire agent systems. ADAS's Meta Agent Search uses a meta-agent to program new agents in code, iteratively archiving and improving designs that can change prompts, tool use, and control flow; discovered agents outperform hand-designed ones and transfer across domains and models [15]. AgentSquare searches a modular space of planning, reasoning, tool-use, and memory modules, reporting an average 17.2% gain over the best human designs on six benchmarks [16]. AFlow represents workflows as code and searches them with Monte Carlo Tree Search driven by execution feedback, reporting a 5.7% average improvement over state-of-the-art baselines and cases where smaller models with optimized workflows outperform GPT-4o at a fraction of its inference cost [17]. Learn-by-Interact synthesizes agent–environment interaction trajectories from documentation without human labels and reuses them as in-context examples or training data across coding, web, and desktop environments [18].

LangProBe evaluates more than 2,000 combinations of language-program architectures, optimizers, tasks, and models, and studies their quality–cost trade-offs [19]. Its central message is that architecture, optimizer, and model choice interact, so they must be compared under a common protocol rather than in isolation.

4.3.5 Critical Comparison and Design Choice

Table 2 contrasts the optimization families by their search space and feedback. Larger search spaces (ADAS, AFlow) can find stronger systems but make it harder to explain *why* behavior changed and easier to alter the task implicitly. Every method can overfit to the examples and metric that guide it, so success must mean improved *held-out* behavior, not a higher development score.

Table 2: Comparison of harness and agent-system optimization families.

Family	Search space	Feedback used	Main limitation for this project
Reflexion [8]	Per-task memory	Verbal self-reflection	Episodic; helps retries, not new tasks.
ProTeGi, OPRO [9, 10]	Single prompt	Errors or scores	Mostly single-call tasks.
DSPy, MIPRO [11, 12]	Instructions + demonstrations	Scalar metric	Little use of trajectory content.
GEPA, SIMBA [13, 14]	Instructions (+ rules)	Reflection on trajectories	Cost beyond rollouts not reported; no routing.
ADAS, AgentSquare, AFlow [15–17]	Code, modules, workflows	Execution scores	Large space; harder attribution and control.
Learn-by-Interact [18]	Synthesized examples	Environment interaction	Data synthesis, not diagnosis-driven edits.

The project therefore uses one reflective optimizer (GEPA) over a bounded textual space, with MIPROv2 as a matched-budget comparison if resources permit. A broad optimizer tournament would dilute the central question, which is whether an adapted harness helps under the same evaluator and whether its benefit persists once routing changes model usage.

4.4 Dynamic Model Routing

4.4.1 Cascades and Escalation

FrugalGPT learns a cascade that queries models in sequence and accepts an answer once a scorer judges it reliable, reporting performance matching the best individual model with up to 98% cost reduction on its tasks [20]. A cascade uses evidence from the generated answer, which can be more informative than the input alone, but pays for earlier attempts and adds latency. For a stateful agent, escalation also risks repeating a side effect such as a refund. Direct task-level selection is therefore the cleaner primary design.

4.4.2 Learned Predictive Routing

RouteLLM learns to choose between a stronger and a weaker model from human preference data with augmentation, reporting more than 2× cost reduction without loss of response quality on public benchmarks and transfer across model pairs [21]. MixLLM extends routing to many models with contextual-bandit assignment, lightweight quality and cost predictors, a latency-aware decision rule, and continual updates, reaching 97.25% of GPT-4's quality at 24.18% of its cost under a time constraint [22]. RouterBench standardizes router evaluation with more than 405,000 inference outcomes and cost–quality curves, and emphasizes comparison against simple and oracle reference routers [23].

These routers optimize *response quality*. The target here is policy-compliant completion of a tool workflow, which preference labels may not predict. Difficulty is also relative to the model

pool: the useful quantity is the expected benefit of the stronger model compared with its extra cost, which is best estimated from per-model execution outcomes.

4.4.3 Routing Inside Agents

Two 2026 preprints evaluate routing in multi-step agents. DecisionBench studies delegation among an eleven-model pool on GAIA, τ-bench, and BFCL multi-turn tasks, reporting quality, cost, latency, and routing fidelity; it finds a gap of 15–31 percentage points between measured performance and a counterfactual perfect-delegation ceiling [24]. TwinRouterBench argues that one-shot router benchmarks miss whether a cheap choice at one step breaks a later step; it provides step-level static labels and a live SWE-bench track, where a logistic router trained on static labels reduced API cost by 53.1% while resolving 75 of 100 held-out cases versus 74 for the unrouted strong model [25]. Both indicate that agentic routing is an open evaluation problem and that oracle ceilings are informative reference points.

Routing granularity. Task-level selection is easy to train and evaluate because a run uses one model. Step-level selection can react to new evidence but introduces credit assignment, context transfer, and repeated-action risks. The primary study uses an offline two-model task router informed by RouteLLM's selection principle, without claiming to reproduce its algorithm. Only input features available before execution are used on held-out tasks.

4.5 Multi-Objective Evaluation

Harness adaptation and routing can move different quantities: a longer prompt may raise success and cost together; cheaper models may increase failed attempts; fewer tool calls may reflect efficiency or premature termination. FrugalGPT and RouteLLM motivate preserving quality while reducing cost, MixLLM adds latency, and LangProBe reports quality–cost frontiers [19–22]. Rather than a weighted sum of incompatible units, this project uses a constrained formulation. Let S, V, C, and L_{95} denote success rate, policy-violation rate, mean cost per attempt, and 95th-percentile latency. A configuration is preferred if it minimizes C subject to

$$S \geq S_{\text{ref}} - \delta, \qquad V \leq v_{\max}, \qquad L_{95} \leq \ell_{\max}, \tag{1}$$

where S_{ref} is the success of the reference configuration and δ an absolute margin in percentage points. These are project design choices, fixed after a development-only pilot and before validation and testing. If no candidate is feasible, that is reported rather than loosening the constraints. A complementary Pareto analysis reports configurations not dominated in success, cost, and latency.

5 Synthesis of Prior Work and Research Gaps

5.1 Cross-Study Comparison

Table 3 summarizes what each core and complementary study contributes. The last column states the boundary relevant to this thesis, not a failure of the paper at a problem it did not set out to solve.

5.2 Research Synthesis

The reviewed studies supply complementary building blocks: benchmarks make success measurable; judge studies show evaluators must be validated; attribution localizes failures; textual and workflow optimizers adapt the system; routers allocate capability. Three relationships between these blocks motivate the proposed experiment.

Table 3: Methodological comparison of reviewed studies, grouped by theme. †2026 work consulted as an arXiv preprint.

Work	Mechanism or evidence	Principal target	Boundary for this project
Evaluation			
τ-bench [1]	Tool–user interaction; final-state checks; pass^k	Task success and consistency	Labels do not indicate which harness change to make.
ToolSandbox [2]	Stateful tools; implicit dependencies; milestones	Conversational tool use	Milestones need annotation; no adaptation studied.
AgentRewardBench [3]	Expert labels; 12 judge configurations	Evaluator reliability	Web domain; judge errors need local validation.
Failure attribution			
Who&When [4]	Attribution benchmark; agent and step labels	Who failed and when	Low step accuracy; no repair measured.
AgentRx [5]	Constraint synthesis; evidence logs	Critical step and category	Diagnosis accuracy $\neq$ useful edits.
DeFA† [6]	Event-dependency graphs	Attribution; skill feedback	Different adaptation pipeline; preprint.
AgentTether† [7]	Graph-guided diagnosis and repair	Runtime recovery	Mainly online intervention.
Harness and system optimization			
Reflexion [8]	Verbal reflection memory	Retry improvement	Episodic, per task.
ProTeGi [9], OPRO [10]	Textual gradients; LLM as optimizer	Single prompts	Single-call tasks.
DSPy [11], MIPRO [12]	Metric-guided compilation; grounded search	LM-program prompts	No routing; scalar feedback.
GEPA [13]	Trajectory reflection; Pareto candidates	Prompt updates	Rollouts $\neq$ dollars; fixed model.
Du et al. [14]	Controlled optimizer comparison	Tabular verification	Single-tool ReAct.
ADAS [15], AgentSquare [16], AFlow [17]	Search over code, modules, workflows	Agent-system design	Broad space; model choice not co-adapted with a router.
Learn-by-Interact [18]	Synthesized trajectories	Environment adaptation	No diagnosis or routing.
LangProBe [19]	2,000+ program/optimizer/model combinations	Quality–cost trade-offs	Mostly non-agentic, no learned router.
Routing			
FrugalGPT [20]	Learned cascade	Query quality vs. cost	Escalation repeats side effects.
RouteLLM [21]	Preference-trained strong/weak router	Query quality–cost	Preferences $\neq$ tool success.
MixLLM [22]	Contextual bandit; latency-aware	Multi-model queries	Response quality only.
RouterBench [23]	405k outcomes; cost–quality curves	Router evaluation	Single-turn queries.
DecisionBench† [24]	Delegation in long-horizon agents	Routing fidelity	Fixed systems; no harness adaptation.
TwinRouterBench† [25]	Step-level static + live tracks	Agentic routing	Fixed harness; coding focus.

Table 4: Selected reported findings from prior work, with context. These are not results of the proposed system, and values are not comparable across rows.

Study	Reported finding	Context
τ-bench [1]	Success $< 50\%$; retail pass$^8 < 25\%$	Original function-calling agents.
AgentRewardBench [3]	Rule F1 67.1% (P 83.8%); GPT-4o judge F1 75.9% (P 69.8%)	Aggregate success judgment.
Who&When [4]	Agent 53.5%, exact step 14.2%	Best method; multi-agent logs.
AgentRx v2 [5]	Exact step $42.7 \pm 1.5\%$ vs. $18.8 \pm 5.3\%$	τ-bench subset; three runs.
MIPRO [12]	Best in 5 of 7 settings, up to +13 pts	Multi-stage LM programs.
GEPA [13]	+6 pts mean vs. GRPO; up to 35× fewer rollouts	Six tasks; rollouts, not dollars.
Du et al. [14]	SIMBA: 32B 61.5→66.2%; 8B 61.3→58.3%	SciTab ReAct.
AFlow [17]	+5.7% average over baselines	Six reasoning/coding benchmarks.
FrugalGPT [20]	Up to 98% cost reduction at matched quality	Maximum over NLP tasks.
RouteLLM [21]	$> 2\times$ cost reduction at equal quality	Response benchmarks.
MixLLM [22]	97.25% of GPT-4 quality at 24.18% cost	Time-constrained queries.
TwinRouterBench† [25]	-53.1% cost; 75/100 vs. 74/100 resolved	Live SWE-bench, held-out.

1. **Diagnosis → adaptation.** Richer traces can enable better edits only if diagnoses are accurate *and* the resulting edits help unseen tasks. Attribution studies mostly measure the first property.
2. **Adaptation ↔ routing.** A better harness can change which model is sufficient for which task, so a router fitted before adaptation may become miscalibrated. Optimization work fixes the model; routing work fixes the system.
3. **Deployment vs. adaptation cost.** Inference savings can be outweighed by the offline cost of finding the configuration, which most studies report only partially.

5.3 Bounded Research Gaps

The broader search shows that the field is already moving toward joint optimization of architecture, workflow, and model choice [15, 17, 19]. The gaps below are therefore stated narrowly; they are inferences from the scopes of the reviewed studies, not claims that no related system exists.

G1 Interaction between harness adaptation and routing. Optimizers are evaluated with fixed models and routers with fixed systems [12, 13, 21, 22]. Agent-system search can change model usage [17, 19] but does not co-adapt a learned task router. Controlled evidence is lacking on whether a router must be re-fitted after harness adaptation and whether the combination improves on each component.

G2 Usefulness of structured diagnosis for offline adaptation. Attribution work measures agreement with human labels [4, 5], and early evidence links diagnosis to improvement under different pipelines [6, 7]. Whether structured, evidence-linked diagnoses improve bounded harness optimization beyond scalar outcome feedback, under a matched budget, remains untested.

G3 Agent-level routing objective. Most routers are trained and assessed on response quality [20–23], whereas stateful agents require end-state correctness, policy compliance, and repeat reliability [1, 2]. Agentic routing benchmarks are emerging [24, 25] but evaluate routing over fixed harnesses and do not report adaptation overhead.

Table 5: Traceability from research gaps to questions, contributions, and experimental evidence.

Gap	RQ	Planned contribution	Evidence that addresses it
G1	RQ3	Controlled A/B/C/D study with a re-fitted router	Interaction I_M; D vs. frozen router; Pareto analysis; break-even.
G2	RQ1	Audited trace-to-edit interface	Diagnostic agreement; scalar vs. structured feedback ablation (B variants).
G3	RQ2	Router trained on policy-compliant success labels	C vs. A, random proportion-matched, and oracle routers; calibration.

6 Proposed Solution and Methodology

6.1 Architecture and Adaptation Boundary

Figure 1 shows the proposed architecture. Adaptation happens offline between batches; the harness and router are frozen during final testing. The optimizer may propose textual changes only; it cannot modify evaluator code, task goals, or policy enforcement.

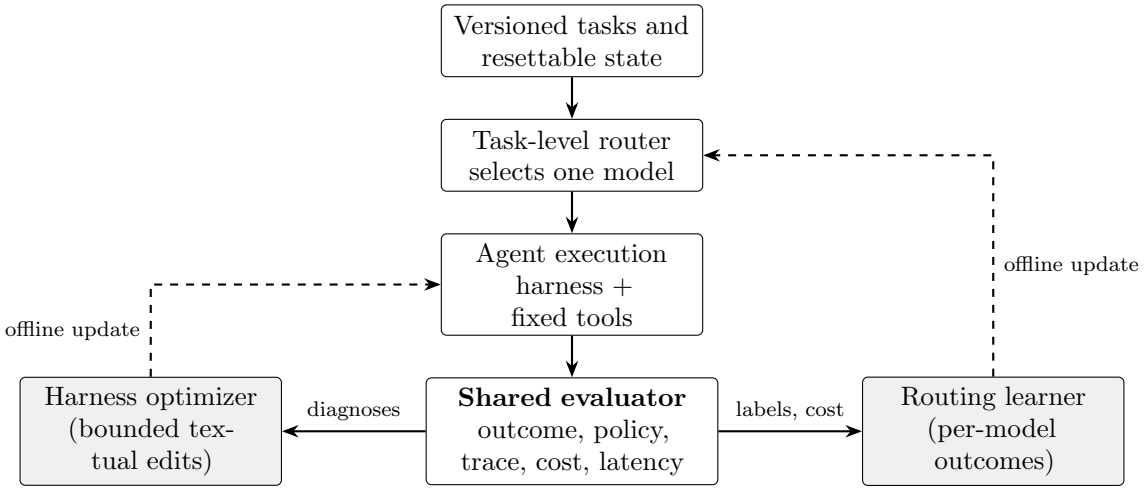


Figure 1: Proposed shared-feedback architecture. Solid arrows show execution; dashed arrows show offline updates applied between batches. Original synthesis informed by [1, 5, 13, 21].

6.2 Benchmark and Baseline Agent

Domain. A sandboxed order-support workflow with about six deterministic tools: inspect customer, inspect order, inspect product, check policy eligibility, update order, and issue an allowed cancellation or refund. Tasks include read-only queries, state-changing requests, and requests that require a policy-compliant refusal or clarification. No real customer data or transactions are used. The design is inspired by τ-bench and ToolSandbox [1, 2] but is not a reproduction of either.

Tasks and splits. The plan is 200 task instances: 100 development, 40 validation, and 60 test. Templates and entity combinations are partitioned *before* any tuning so that paraphrases of one underlying task never cross splits. Requests and clarifications are scripted to avoid dependence on a stochastic simulated user. Counts are targets, adjustable only through a documented development-only pilot.

Models and baseline. The pool contains one stronger model (m_s) and one cheaper model (m_c) from the same API family where possible. The baseline harness is written manually. All configurations share the tools, step limit, temperature, and timeout recorded in an experiment manifest.

External-validity check (optional). If budget allows, configurations A and D are additionally run on a small τ-bench retail subset to test whether the direction of the main finding transfers beyond the authored benchmark. This is a secondary check, not a core deliverable.

6.3 Evaluation Engine and Diagnostic Audit

Each run receives a deterministic goal-success label and a separate policy-violation label. Schema checks detect malformed arguments; tool preconditions detect invalid operations; state invariants detect unauthorized side effects. Valid alternative action orders are accepted. Tool correctness is reported at two levels: *syntactic* validity (schema conformance) and *semantic* validity (the operation is allowed in the current state for this task). Checkers are unit-tested on fixtures covering success, failure, valid alternative paths, and policy violations.

The diagnostic layer proposes a step, category, and evidence for each failed run, or returns *inconclusive*. Two reviewers independently label a stratified audit sample of development and validation traces (minimum 40, target 80), with disagreements adjudicated; the sample and instructions are fixed before the diagnostic configuration is chosen. We report exact and ± 1-step localization accuracy, per-category precision, recall, and F1 with support, inconclusive rate, and Cohen's κ. The goal checker always remains the source of success labels.

6.4 Harness Adaptation

GEPA performs a bounded reflective search over instructions, demonstrations, and tool descriptions using development traces and the strong model m_s. Its objective prioritizes policy-compliant success, with cost as a tie-breaker among acceptable candidates. Every candidate records its parent, the feedback used, the edit, and its measured outcome. Prompt length is capped, and the original harness is retained if no candidate improves validation behavior. Validation selects the final harness; test data are never visible to the optimizer.

Two secondary comparisons use the same edit space and a matched *monetary* cap: (i) GEPA with scalar outcome feedback only versus GEPA with structured diagnoses (addresses **G2**); (ii) MIPROv2 versus GEPA. These variants are selected on validation and compared at the A/B level only. The core four-configuration study needs only the single selected harness, so dropping either comparison for budget reasons does not affect the main research question.

6.5 Offline Routing Policy

Training labels. Every development task is executed with both models under the relevant harness, three times each, giving an estimated success probability per task and model rather than a single noisy binary label. This collection cost is counted as routing adaptation cost.

Predictor and rule. Features are computable from the input alone: task category, input length, number of stated constraints, and a sentence embedding. Hidden goal state, future outcomes, and test labels are excluded. A regularized logistic model estimates $\hat{p}_m(x)$, the probability of policy-compliant success for model m, and $\hat{c}_m(x)$ is estimated from development costs per category. The router chooses

$$r(x) = \arg\min_{m \in \mathcal{M}} \hat{c}_m(x) \quad \text{subject to} \quad \hat{p}_m(x) \geq q_{\min}, \tag{2}$$

defaulting to m_s (and logging a low-confidence decision) when no model satisfies the threshold. $q_{\min}$ is selected on validation data under Eq. (1), and calibration is checked there with the Brier score and a reliability diagram.

Re-fitting. For C the router is trained on fixed-harness development runs; for D it is re-trained on development runs produced by the selected adapted harness, because adaptation may change relative model performance.

6.6 Four-Configuration Experiment

Table 6: Core configurations. All use identical tasks, tools, evaluator, and execution limits.

ID	Harness	Model strategy	Purpose
A	Fixed (manual)	Fixed strong model m_s	Baseline quality and cost.
B	Optimized	Fixed strong model m_s	Isolate harness adaptation.
C	Fixed (manual)	Router fitted on fixed harness	Isolate routing.
D	Optimized (as in B)	Router re-fitted on optimized harness	Combined pipeline.

Efficient execution grid. Because the router fixes one model per run, the outcome of a routed run depends only on the harness–model pair chosen. The test protocol therefore runs four *fixed* cells—{fixed, optimized} harness $\times$ $\{m_s, m_c\}$—plus C and D directly (Figure 2). The fixed cells give A and B directly and two cheap-model controls (A_c, B_c). The direct C and D runs measure routing overhead and real latency.

Controls. The cheap-only cells A_c and B_c are run directly and test whether routing beats simply using m_c everywhere. The following routing controls need no extra API calls; they are computed by applying each policy's per-task choices to the observed fixed-cell runs of the same harness:

- **Random proportion-matched router:** sends the same fraction of tasks to m_s as C (or D) but chooses them at random (averaged over 1,000 draws). If C is no better, routing savings come from strong-model usage rate, not task-conditioned selection.
- **Oracle router:** per task, the cheapest model whose observed success rate is highest. It bounds the attainable routing gain; we report the fraction of oracle gain that C and D achieve [23, 24].
- **Frozen router (E):** C's router applied to the optimized harness. Comparing D with E tests whether harness adaptation invalidates a previously learned router (**G1**).

Derived controls use the same runs as the configurations they are compared with, so their comparison is paired. We also check agreement between direct C/D runs and their derived counterparts as a sanity test of this design.

Workload. With 60 test tasks and five repetitions, the four fixed cells require 1,200 executions and direct C and D runs a further 600, for 1,800 test executions. Router training requires about 1,200 development executions (100 tasks $\times$ 2 models $\times$ 3 repeats $\times$ 2 harnesses). These are planning estimates, revised by the pilot.

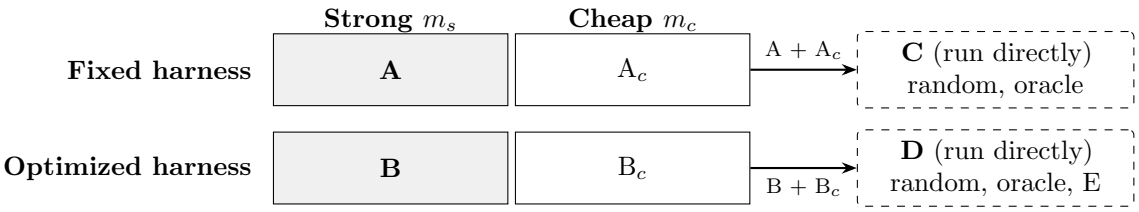


Figure 2: Test execution grid. Shaded cells are core configurations run directly; with C and D, they form the four-configuration study. Routing controls are derived from the fixed cells of the same harness row.

Interaction. For metric M, the descriptive interaction is

$$I_M = (M_D - M_C) - (M_B - M_A). \tag{3}$$

For success, $I_M > 0$ means the harness benefit is larger within the routed pipeline. Because D includes re-fitting the router, I_M describes an interaction between adaptation *procedures*, not a purely mechanistic effect.

Hypotheses. The study tests, without presupposing, that: **H1** B improves policy-compliant success over A; **H2** C reduces cost per attempt relative to A within margin δ and outperforms the random proportion-matched router; **H3** D is feasible under Eq. (1) at lower cost than B and C; **H4** D outperforms the frozen router E. Each may be rejected or remain inconclusive.

7 Technical Requirements and Reproducibility

The minimum implementation is a Python execution loop, a deterministic tool service, a structured trace store, the evaluator, a harness-optimizer adapter (DSPy/GEPA), the router, and an experiment runner. A small explicit loop is sufficient; an orchestration framework is optional and is not a research contribution.

The runner resets state for every attempt, enforces tool schemas and the step limit, and distinguishes task failure, model timeout, tool error, and infrastructure error. Failed and invalid attempts are retained so cost and latency are not conditioned on success. Each run record links task, initial state, harness hash, router version, model identifier, outputs, and resources. Every manifest records software versions, prompt hashes, seeds where supported, model settings, timeouts, split identifiers, and a dated price schedule. One command regenerates all summaries from immutable run records; raw traces and derived metrics are stored separately so evaluator changes can be audited.

Table 7: Indicative work plan for Graduation Project II.

Phase	Work	Exit criterion
1	Benchmark, tools, checkers, runner	Fixture tests pass; A runs end to end.
2	Pilot on development split	Budgets, δ, $v_{\max}$, $\ell_{\max}$, repetitions frozen.
3	Diagnosis layer and human audit	Audit metrics reported.
4	Harness optimization (+ ablations if funded)	Harness selected on validation.
5	Router data collection and training (C, D)	Routers and $q_{\min}$ frozen on validation.
6	Test runs and analysis	All intervals and failure analyses reported.

8 Experimental Validity and Analysis Plan

8.1 Data Separation and Fair Selection

Development data drive optimizer proposals and router labels; validation data select candidates, thresholds, and calibration; test tasks are accessed once, after every choice is frozen (Figure 3). The pilot, run on development data only, fixes the number of repetitions, the optimization budget, and the margins in Eq. (1). Any change to task or repetition counts is documented before testing. If the study is too small to distinguish acceptable from unacceptable loss, the result is reported as inconclusive.

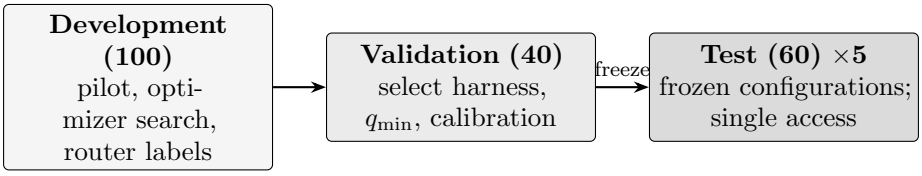


Figure 3: Data separation. Templates are partitioned before tuning; information flows only left to right.

8.2 Metric Definitions and Resource Accounting

For N attempts, let s_i be goal success, v_i an indicator of any policy violation, c_i the monetary cost, and l_i end-to-end wall-clock latency. The primary quality outcome is *policy-compliant success*, $s_i(1 - v_i)$. Table 8 lists all metrics; the main summaries are

$$\hat{S} = \frac{1}{N}\sum_{i=1}^{N} s_i(1 - v_i), \quad \hat{V} = \frac{1}{N}\sum_{i=1}^{N} v_i, \quad \bar{C} = \frac{1}{N}\sum_{i=1}^{N} c_i, \quad C_{\text{succ}} = \frac{\sum_i c_i}{\sum_i s_i(1 - v_i)}. \tag{4}$$

C_{succ} includes the cost of failed attempts and is reported as undefined when no attempt succeeds. Latency runs from task start to termination, including routing, model and tool calls, and in-run checks; median and 95th percentile are reported. Tokens are priced under one dated schedule, including cached-token charges.

Table 8: Metrics by dimension. Primary metrics drive the hypotheses; secondary metrics explain them.

Dimension	Primary	Secondary / diagnostic
Task success	Policy-compliant success $\hat{S}$	Raw goal success; syntactic and semantic tool validity
Policy	Violation rate $\hat{V}$	Violations by type
Cost	$\bar{C}$ per attempt; C_{succ}	Tokens; prompt length; strong-model share
Latency	p_{50} and p_{95} end-to-end	Steps and tool calls per run
Reliability	Fraction of tasks succeeding in all 5 runs	Per-task success variance
Routing	Gain vs. random router; % of oracle gain	Brier score; selection regret
Adaptation	Offline cost K; break-even n	Optimizer calls, candidates, wall-clock

Adaptation overhead. Offline cost K includes diagnosis, reflection and proposal calls, candidate evaluation, and dual-model router data collection. With $\Delta c > 0$ the reduction in mean cost per future attempt relative to A,

$$n_{\text{break-even}} = \lceil K/\Delta c \rceil. \tag{5}$$

This is meaningful only for a feasible configuration and a stable task mix; if $\Delta c \leq 0$, no cost break-even is claimed. Reporting it keeps one-time optimization expense visible.

8.3 Repeated Runs, Uncertainty, and Failure Reporting

All configurations run on the same test tasks from identical reset states, with task order interleaved across configurations and fixed concurrency to limit temporal API effects. A timeout after execution starts is a failed attempt with its observed cost and time; infrastructure errors before the agent runs follow a predeclared retry policy and are listed separately.

Uncertainty is estimated with a *paired cluster bootstrap* over task identifiers (10,000 resamples), keeping all repetitions of a sampled task and pairing configurations. We report 95% intervals for differences rather than isolated significance tests. Primary contrasts are B−A, C−A, and D−A; D−B, D−C, D−E, and C−random are component comparisons. Repeated attempts are never treated as independent tasks. Reliability is summarized as the fraction of tasks succeeding in all five runs, distinguished from a precise estimate of per-task reliability. Results are presented as a success–cost scatter with Pareto front and intervals, a table of contrasts, and failure-category counts with trace examples. No numerical result will be inserted until the corresponding runs are complete.

Threats to validity. *Internal:* evaluator bugs or omitted invariants (mitigated by fixtures and inspection of accepted traces); leakage across splits (mitigated by template partitioning). *Construct:* cost depends on a dated price schedule (both billed and normalized costs retained). *External:* one authored domain and two models (optional τ-bench subset check). *Statistical:* 60 test tasks may not detect small effects (intervals reported, inconclusive results stated as such).

9 Expected Contribution and Positioning

The intended contribution is a *controlled empirical study* of existing methods integrated around one evaluation contract, not a new optimizer or router. Its concrete outputs are: (i) a resettable, policy-constrained tool benchmark with validated checkers; (ii) versioned execution and audited failure records; (iii) a bounded, diagnosis-informed harness-adaptation procedure; (iv) an offline task router trained on policy-compliant success; and (v) the four-configuration comparison with derived routing controls and adaptation accounting. Table 5 maps these outputs to **G1**–**G3**.

Relative to agent-system search such as ADAS and AFlow [15, 17], the study searches a deliberately narrower space but adds a co-adapted router and explicit adaptation-cost accounting. Relative to agentic routing benchmarks [24, 25], it studies how routing interacts with a *changing* harness. A negative or inconclusive result remains informative if it identifies router–harness mismatch, evaluator weakness, or an adaptation cost that prevents practical savings.

10 Limitations and Risks

The review is focused rather than systematic and cannot establish exhaustive novelty; several 2026 sources are preprints and are used only as context. Reported gains in prior work come from different tasks and model generations and do not predict this project's effect sizes. One authored domain limits external validity: findings may not transfer to browsing, coding, or long-horizon autonomous work.

Automatic diagnosis may confuse symptoms with causes, and an audit of 40–80 traces gives limited precision for rare categories, so per-category support is always reported. API version changes, price changes, small test size, and stochastic behavior can obscure modest effects; model versions and run dates are recorded. A harness optimized with the strong model may transfer poorly to the cheaper one; the B_c cell and configuration D measure this directly. Finally, a two-model task router leaves step-level escalation and larger pools for future work.

11 Conclusion

The reviewed literature supports an evaluation chain from measurable outcomes to failure attribution, harness adaptation, and model selection. It also shows why each stage needs its own validation: success checks can be incomplete, judges make systematic errors, attribution is hard at the step level, optimizer gains depend on configuration, and routers trained on response quality may not predict tool success. The field is moving toward joint optimization of agent systems, which makes careful positioning essential.

This project turns these findings into a bounded offline experiment. A shared evaluator feeds a reflective harness optimizer and a task-level router, and four configurations, supported by random, oracle, and frozen-router controls, test their separate and combined effects on unseen tasks. The question to be answered in Graduation Project II is whether integration yields a feasible and better success–cost–latency trade-off once adaptation overhead is counted; this document establishes the literature basis and the protocol, not the answer.

References

[1] S. Yao, N. Shinn, P. Razavi, and K. Narasimhan. τ-bench: A benchmark for tool-agent-user interaction in real-world domains. In *International Conference on Learning Representations (ICLR)*, 2025. https://proceedings.iclr.cc/paper_files/paper/2025/hash/1b126cc38b8638e07bef37e7b2bb72bf-Abstract-Conference.html.

[2] J. Lu, T. Holleis, Y. Zhang, B. Aumayer, F. Nan, F. Bai, S. Ma, S. Ma, M. Li, G. Yin, Z. Wang, and R. Pang. ToolSandbox: A stateful, conversational, interactive evaluation benchmark for LLM tool use capabilities. In *Findings of the Association for Computational Linguistics: NAACL*, 2025. doi:10.18653/v1/2025.findings-naacl.65.

[3] X. H. Lù, A. Kazemnejad, N. Meade, A. Patel, D. Shin, A. Zambrano, K. Stańczak, P. Shaw, C. J. Pal, and S. Reddy. AgentRewardBench: Evaluating automatic evaluations of web agent trajectories. arXiv:2504.08942v2, 2025.

[4] S. Zhang, M. Yin, J. Zhang, J. Liu, Z. Han, J. Zhang, B. Li, C. Wang, H. Wang, Y. Chen, and Q. Wu. Which agent causes task failures and when? On automated failure attribution of LLM multi-agent systems. In *International Conference on Machine Learning (ICML)*, PMLR 267, 2025. https://proceedings.mlr.press/v267/zhang25cq.html.

[5] S. Barke, A. Goyal, A. Khare, A. Singh, S. Nath, and C. Bansal. AgentRx: Diagnosing AI agent failures from execution trajectories. arXiv:2602.02475v2, 2026. To appear in *Proceedings of EMNLP 2026*; figures in this thesis are taken from arXiv v2.

[6] B. Deng, X. Zheng, Y. Wei, K. Zhou, C. Tao, R. Liang, X. Chen, L. Guo, and C. Zhang. DeFA: Dependency-guided failure attribution for LLM agents. arXiv:2610.01256, preprint, 2026.

[7] C. Zhao, S. Zhang, W. Gu, Y. Sun, D. Pei, C. Bansal, S. Rajmohan, and M. Ma. AgentTether: Graph-guided diagnosis and runtime intervention for reliable LLM agent operation. arXiv:2607.06273, preprint, 2026.

[8] N. Shinn, F. Cassano, A. Gopinath, K. Narasimhan, and S. Yao. Reflexion: Language agents with verbal reinforcement learning. In *Advances in Neural Information Processing Systems (NeurIPS)*, 2023. https://proceedings.neurips.cc/paper_files/paper/2023/hash/1b44b878bb782e6954cd888628510e90-Abstract-Conference.html.

[9] R. Pryzant, D. Iter, J. Li, Y. T. Lee, C. Zhu, and M. Zeng. Automatic prompt optimization with "gradient descent" and beam search. In *Proceedings of EMNLP*, 2023. doi:10.18653/v1/2023.emnlp-main.494.

[10] C. Yang, X. Wang, Y. Lu, H. Liu, Q. V. Le, D. Zhou, and X. Chen. Large language models as optimizers. In *International Conference on Learning Representations (ICLR)*, 2024. https://proceedings.iclr.cc/paper_files/paper/2024/hash/3339f19c5fcee3ad74502947a32be9e6-Abstract-Conference.html.

[11] O. Khattab, A. Singhvi, P. Maheshwari, Z. Zhang, K. Santhanam, S. Vardhamanan A, S. Haq, A. Sharma, T. Joshi, H. Moazam, H. Miller, M. Zaharia, and C. Potts. DSPy: Compiling declarative language model calls into state-of-the-art pipelines. In *International Conference on Learning Representations (ICLR)*, 2024. https://proceedings.iclr.cc/paper_files/paper/2024/hash/f1cf02ce09757f57c3b93c0db83181e0-Abstract-Conference.html.

[12] K. Opsahl-Ong, M. J. Ryan, J. Purtell, D. Broman, C. Potts, M. Zaharia, and O. Khattab. Optimizing instructions and demonstrations for multi-stage language model programs. In *Proceedings of EMNLP*, pp. 9340–9366, 2024. doi:10.18653/v1/2024.emnlp-main.525.

[13] L. A. Agrawal, S. Tan, D. Soylu, N. Ziems, R. Khare, K. Opsahl-Ong, A. Singhvi, H. Shandilya, M. J. Ryan, M. Jiang, C. Potts, K. Sen, A. Dimakis, I. Stoica, D. Klein, M. Zaharia, and O. Khattab. GEPA: Reflective prompt evolution can outperform reinforcement learning. In *International Conference on Learning Representations (ICLR)*, 2026. https://proceedings.iclr.cc/paper_files/paper/2026/hash/0e9e708b6f48e14fd0ac29e167413f76-Abstract-Conference.html.

[14] X. Du, G. Hong, W.-C. Kwan, R. Saxena, I. Titov, P. Minervini, and E. Allaway. Analyzing LLM instruction optimization for tabular fact verification. In *Findings of the Association for Computational Linguistics: EACL*, pp. 3078–3108, 2026. doi:10.18653/v1/2026.findings-eacl.161.

[15] S. Hu, C. Lu, and J. Clune. Automated design of agentic systems. In *International Conference on Learning Representations (ICLR)*, 2025. https://proceedings.iclr.cc/paper_files/paper/2025/hash/36b7acf6f6010652b3f2a433774a66fe-Abstract-Conference.html.

[16] Y. Shang, Y. Li, K. Zhao, L. Ma, J. Liu, F. Xu, and Y. Li. AgentSquare: Automatic LLM agent

search in modular design space. In *International Conference on Learning Representations (ICLR)*, 2025. https://openreview.net/forum?id=mPdmDYIQ7f.

[17] J. Zhang, J. Xiang, Z. Yu, F. Teng, X. Chen, J. Chen, M. Zhuge, X. Cheng, S. Hong, J. Wang, B. Zheng, B. Liu, Y. Luo, and C. Wu. AFlow: Automating agentic workflow generation. In *International Conference on Learning Representations (ICLR)*, 2025. https://proceedings.iclr.cc/paper_files/paper/2025/hash/5492ecbce4439401798dcd2c90be94cd-Abstract-Conference.html.

[18] H. Su, R. Sun, J. Yoon, P. Yin, T. Yu, and S. Ö. Arık. Learn-by-Interact: A data-centric framework for self-adaptive agents in realistic environments. In *International Conference on Learning Representations (ICLR)*, 2025. https://proceedings.iclr.cc/paper_files/paper/2025/hash/dd745a0c4f91fe91866fe6788be9cc28-Abstract-Conference.html.

[19] S. Tan, L. A. Agrawal, A. Singhvi, L. Lai, M. J. Ryan, D. Klein, O. Khattab, K. Sen, and M. Zaharia. LangProBe: A language programs benchmark. In *Findings of the Association for Computational Linguistics: EMNLP*, 2025. doi:10.18653/v1/2025.findings-emnlp.1172.

[20] L. Chen, M. Zaharia, and J. Zou. FrugalGPT: How to use large language models while reducing cost and improving performance. *Transactions on Machine Learning Research*, 2024. https://openreview.net/forum?id=cSimKw5p6R.

[21] I. Ong, A. Almahairi, V. Wu, W.-L. Chiang, T. Wu, J. E. Gonzalez, M. W. Kadous, and I. Stoica. RouteLLM: Learning to route LLMs from preference data. In *International Conference on Learning Representations (ICLR)*, 2025. https://proceedings.iclr.cc/paper_files/paper/2025/hash/5503a7c69d48a2f86fc00b3dc09de686-Abstract-Conference.html.

[22] X. Wang, Y. Liu, W. Cheng, X. Zhao, Z. Chen, W. Yu, Y. Fu, and H. Chen. MixLLM: Dynamic routing in mixed large language models. In *Proceedings of NAACL: Human Language Technologies, Volume 1*, pp. 10912–10922, 2025. doi:10.18653/v1/2025.naacl-long.545.

[23] Q. J. Hu, J. Bieker, X. Li, N. Jiang, B. Keigwin, G. Ranganath, K. Keutzer, and S. K. Upadhyay. RouterBench: A benchmark for multi-LLM routing system. arXiv:2403.12031, 2024.

[24] Y. Gao, M. Wang, Y. L. Yu, Z. C. Ma, and A. Qu. DecisionBench: A benchmark for emergent delegation in long-horizon agentic workflows. arXiv:2605.19099, preprint, 2026.

[25] P. Yang, W. Chen, T. Yang, P. Feng, J. Xing, W. Guo, Y. Yao, Y. Han, H. Li, X. Wang, Z. Wang, J. Xiao, A. Yang, L. Tian, L. Ai, E. Yang, and T. Shi. TwinRouterBench: Fast static and live dynamic evaluation for realistic agentic LLM routing. arXiv:2605.18859v2, 2026. Listed as a NeurIPS 2026 poster; preprint version consulted.

[26] DSPy contributors. DSPy optimizers: MIPROv2, SIMBA, and GEPA. Software documentation (version 3.2.0), accessed 16 October 2026. https://dspy.ai/3.2.0/learn/optimization/optimizers/.